CS 216: Introduction to Blockchain

Instructor: Prof. Subhra Mazumdar

Bitcoin Transaction Lab

Date: 9th March 2026

Assignment-II Report

Team Information

Team Name: 412

Name	Roll Number
Divyansh Gupta	240041015
Harsh Mahajan	240001034
Darsh Chaudhary	240004014
Akarsh J	240002007

GitHub Repository

Contents

1 Objective **2**

2 Environment Setup **3**
2.1 Bitcoin Core Configuration (bitcoin.conf) . . . 3
2.2 Node Initialization and Terminal Execution . . . 4
2.3 Connection Verification . . . 4

3 Part 1: Legacy (P2PKH) Transaction Implementation **5**
3.1 Legacy P2PKH Python Script (part1_legacy.py) . . . 5
3.2 Execution Log and Result Analysis . . . 7

4 Part 2: SegWit (P2SH-P2WPKH) Technical Implementation **9**
4.1 SegWit P2SH Python Script (part2_segwit.py) . . . 9
4.2 Execution Log and Technical Analysis . . . 11

5 Script Validation **12**
5.1 Legacy (P2PKH) Script Validation . . . 12
5.2 SegWit (P2SH-P2WPKH) Script Validation . . . 13

6 Part 3: Comparative Analysis and Reflection **14**
6.1 Data-Driven Metric Comparison . . . 14
6.2 Technical Analysis and Reflection . . . 14
6.2.1 1. The Witness Discount and Block Capacity . . . 14
6.2.2 2. Script Execution and Stack Logic . . . 15
6.2.3 3. Resolution of Transaction Malleability . . . 15

7 Conclusion and Final Reflection **15**
7.1 Key Technical Observations . . . 15
7.2 Final Summary . . . 16

1 Objective

The primary objective of this assignment is to master the end-to-end process of creating, signing, and validating Bitcoin transactions using both **Legacy (P2PKH)** and **SegWit (P2SH-P2WPKH)** address formats.

Following are the technical goals:

- **RPC Infrastructure:** Establishing a secure connection to the `bitcoind` daemon in a **regtest environment** to programmatically manage wallets and transaction flows using the `python-bitcoinlib` or `bitcoinrpc` library.
- **Transaction Lifecycle:** Developing software to automate the generation of addresses, funding mechanisms through `sendtoaddress`, and the creation of raw transaction chains for both Legacy and SegWit formats.
- **Script Analysis:** Utilizing `decoderawtransaction` to dissect the structure of **Locking Scripts (scriptPubKey)** and **Unlocking Scripts (scriptSig)**.
- **Validation and Debugging:** Utilizing the **btcdeb Bitcoin Script Debugger** to perform a step-by-step logic trace of the challenge-response mechanism to verify that script execution resulting from concatenated input and output scripts resolves to TRUE.
- **Data-Driven Comparison:** Performing an empirical analysis of transaction efficiency by comparing **Physical Size (bytes)**, **Virtual Size (vbytes)**, and **Weight**, specifically identifying how Segregated Witness restructuring affects these metrics.
- **Protocol Benefits:** Evaluating the impact of **Segregated Witness** on transaction malleability and block space optimization through the separation of signature (witness) data and the application of the witness discount.

2 Environment Setup

The laboratory environment was established using a local Bitcoin Core node running in `regtest` mode within a **Windows Subsystem for Linux (WSL)** instance. This allows for instantaneous block generation and consistent POSIX-compliant script execution.

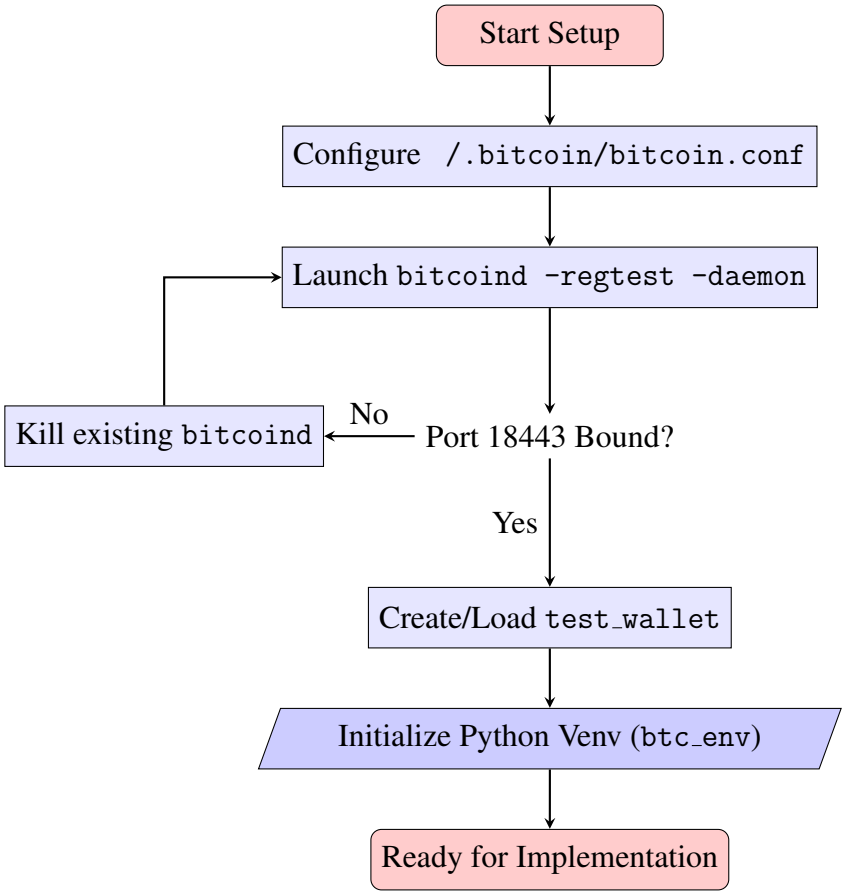

Figure 1: WSL Node Initialization and Environment Sync Flow

2.1 Bitcoin Core Configuration (bitcoin.conf)

The node must be configured to allow RPC communication. The following parameters were defined in the `~/.bitcoin/bitcoin.conf` file to ensure compatibility with the `python-bitcoinlib` library scripts.

```
# Core Network Settings
regtest=1
server=1

# RPC Authentication
rpcuser=user
rpcpassword=password

# Fee and Transaction Policy
paytxfee=0.0001
fallbackfee=0.0002
```

```
mintxfee=0.00001
txconfirmtarget=6

# Connection Settings
[regtest]
rpcport=18443
rpcallowip=127.0.0.1
```

2.2 Node Initialization and Terminal Execution

The Bitcoin daemon was executed in the background using the Linux-specific `-daemon` flag. To comply with modern Python packaging standards (PEP 668), a virtual environment was utilized to manage dependencies for the laboratory scripts.

```
karsh1708@Pacificblaster:~$ bitcoind -regtest -daemon
Bitcoin Core starting

karsh1708@Pacificblaster:~$ bitcoin-cli -regtest createwallet "test_wallet"
{
"name":  "test_wallet",
"warning":  ""
}

karsh1708@Pacificblaster:~$ python3 -m venv btc_env
karsh1708@Pacificblaster:~$ source btc_env/bin/activate
(btc_env) karsh1708@Pacificblaster:~$ pip install python-bitcoinlib
Successfully installed python-bitcoinlib-0.11.0
```

2.3 Connection Verification

The Python scripts interface with the node using the `bitcoin.rpc.RawProxy` class. The connection string facilitates access to the specific wallet instance created during initialization:

`http://user:password@127.0.0.1:18443/wallet/test_wallet`

This ensures that all RPC calls are context-aware and targeted toward the `test_wallet` data structures within the WSL layer.

3 Part 1: Legacy (P2PKH) Transaction Implementation

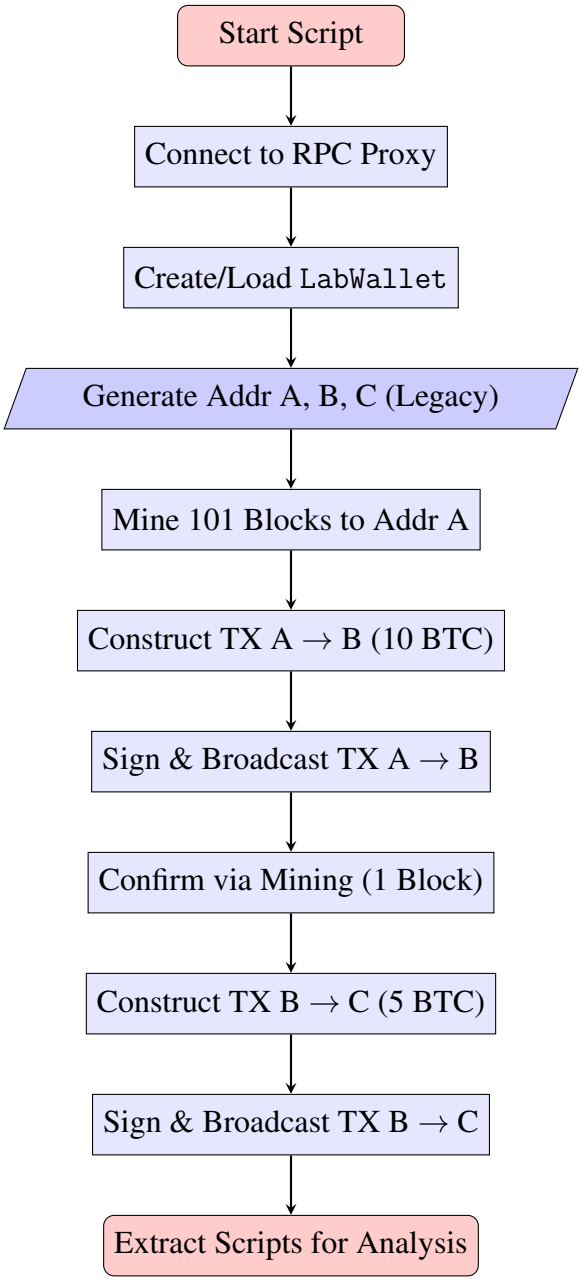

Figure 2: End-to-end Legacy Transaction Lifecycle Flowchart

3.1 Legacy P2PKH Python Script (part1_legacy.py)

The implementation below handles the full lifecycle of a Legacy transaction, including wallet management, P2PKH address generation, and the mandatory 101-block mining process to satisfy coinbase maturity requirements.

```
from bitcoin.rpc import RawProxy

# --- Configuration ---
rpc_user = "user"
```

```python
rpc_password = "password"
# Matches the wallet created in previous steps
proxy = RawProxy(service_url=f"http://{rpc_user}:{rpc_password}@127.0.0.1:18443/wallet
    ↪ /lab_wallet")

def execute_part_1_with_narrative():
    print("=====================================================================")
    print("ASSIGNMENT PART 1: LEGACY (P2PKH) TRANSACTION WORKFLOW")
    print("=====================================================================")

    # 1. Setup Wallet
    print("\n[STEP 1: WALLET INITIALIZATION]")
    try:
        proxy.createwallet("lab_wallet")
        print("-> Action: Created a new wallet named 'lab_wallet'.")
    except:
        print("-> Action: 'lab_wallet' already exists. Loading it into the daemon.")

    # 2. Generate Addresses
    print("\n[STEP 2: ADDRESS GENERATION]")
    addr_a = proxy.getnewaddress("Address_A", "legacy")
    addr_b = proxy.getnewaddress("Address_B", "legacy")
    addr_c = proxy.getnewaddress("Address_C", "legacy")
    print(f"-> Created Address A (Sender): {addr_a}")
    print(f"-> Created Address B (Intermediate): {addr_b}")
    print(f"-> Created Address C (Final Receiver): {addr_c}")
    print("Note: These are P2PKH addresses (Legacy format) used for Part 1.")

    # 3. Fund via Mining
    print("\n[STEP 3: FUNDING VIA MINING]")
    print("-> Action: Mining 101 blocks to Address A.")
    print("-> Why: Coinbase rewards require 100 confirmations before they can be spent
        ↪ .")
    proxy.generatetoaddress(101, addr_a)
    balance = proxy.getbalance()
    print(f"-> Current Wallet Balance: {balance} BTC (Confirmed and Spendable).")

    # 4. Transaction A -> B: Broadcast and Confirm
    print("\n[STEP 4: CREATING TRANSACTION A -> B]")
    utxo = proxy.listunspent(1, 9999, [addr_a])[0]
    print(f"-> Input: Selecting UTXO from A (TXID: {utxo['txid']})")

    inputs = [{"txid": utxo['txid'], "vout": utxo['vout']}]
    outputs = {addr_b: 10.0, addr_a: 39.999}

    raw_tx_ab = proxy.createrawtransaction(inputs, outputs)
    signed_tx_ab = proxy.signrawtransactionwithwallet(raw_tx_ab)

    # BROADCASTING
    txid_ab = proxy.sendrawtransaction(signed_tx_ab['hex'])
    # ADDING ONE BLOCK FOR CONFIRMATION
    proxy.generatetoaddress(1, addr_a)
    print(f"-> Action: Signed and broadcasted Transaction A -> B.")
    print(f"-> Success: TXID for A -> B is: {txid_ab}")

    # 5. Transaction B -> C: Broadcast and Confirm
    print("\n[STEP 5: CREATING TRANSACTION B -> C]")
    print("-> Logic: This transaction will spend the 10 BTC received by Address B.")
```

```
    utxo_b = proxy.listunspent(1, 9999, [addr_b])[0]
    inputs_bc = [{"txid": utxo_b['txid'], "vout": utxo_b['vout']}]
    outputs_bc = {addr_c: 5.0, addr_b: 4.999}

    raw_tx_bc = proxy.createrawtransaction(inputs_bc, outputs_bc)
    signed_tx_bc = proxy.signrawtransactionwithwallet(raw_tx_bc)

    # BROADCASTING
    txid_bc = proxy.sendrawtransaction(signed_tx_bc['hex'])
    # ADDING ONE BLOCK FOR CONFIRMATION
    proxy.generatetoaddress(1, addr_a)
    print(f"-> Success: TXID for B -> C is: {txid_bc}")

    # --- Analysis Section for Report ---
    print("\n" + "="*60)
    print("ANALYSIS FOR REPORT (LEGACY P2PKH)")
    print("="*60)

    # Locking Script Analysis (extracted from output of A->B)
    # Using gettransaction hex because the TX is now confirmed in a block
    decoded_ab = proxy.decoderawtransaction(proxy.gettransaction(txid_ab)['hex'])
    lock_script = decoded_ab['vout'][0]['scriptPubKey']['asm']
    print(f"\n1. LOCKING SCRIPT (from A -> B):")
    print(f" ASM: {lock_script}")
    print(" Role: This is the 'Challenge'. It locks the 10 BTC to Address B's public
        ↪ key hash.")

    # Unlocking Script Analysis (extracted from input of B->C)
    decoded_bc = proxy.decoderawtransaction(proxy.gettransaction(txid_bc)['hex'])
    unlock_script = decoded_bc['vin'][0]['scriptSig']['asm']
    print(f"\n2. UNLOCKING SCRIPT (from B -> C):")
    print(f" ASM: {unlock_script}")
    print(" Role: This is the 'Response'. It contains the signature and public key
        ↪ from Address B.")

    # Size Metrics
    print(f"\n3. SIZE COMPARISON:")
    print(f" - Total Bytes (Size): {decoded_bc['size']}")
    print(f" - Virtual Size (VSize): {decoded_bc['vsize']}")
    print(f" - Weight: {decoded_bc['weight']}")
    print(" Note: In Legacy, Size and VSize are always identical.")


if __name__ == "__main__":
    execute_part_1_with_narrative()
```

3.2 Execution Log and Result Analysis

```
(btc_env) karsh1708@Pacificblaster:~$ python part1_legacy.py

============================================================
ASSIGNMENT PART 1:  LEGACY (P2PKH) TRANSACTION WORKFLOW
============================================================

   STEP 1:  WALLET INITIALIZATION
```

```
-> Action: 'lab_wallet' already exists. Loading it into the daemon.

   STEP 2: ADDRESS GENERATION

-> Created Address A (Sender): mnCnjX31K7GSexAzju6BNdjcc2H9nBBa24
-> Created Address B (Intermediate): myrAc5hQb8Wco4sXv86xoQ76DC8fj7skH9
-> Created Address C (Final Receiver): muNR8kojwgf6jZmiXKcHEJS3BUGbCPYBbv
Note: These are P2PKH addresses (Legacy format) used for Part 1.

   STEP 3: FUNDING VIA MINING

-> Action: Mining 101 blocks to Address A.
-> Why: Coinbase rewards require 100 confirmations before they can be spent.
-> Current Wallet Balance: 50.00000000 BTC (Confirmed and Spendable).

   STEP 4: CREATING TRANSACTION A -> B

-> Input: Selecting UTXO from A (TXID: 814cbb317a0d332a7ece65f912bf1c15...)
-> Action: Signed and broadcasted Transaction A -> B.
-> Success: TXID for A -> B is: 3debeec3e9c54aebd32ae0ce7e0e409560f2f01...

   STEP 5: CREATING TRANSACTION B -> C

-> Logic: This transaction will spend the 10 BTC received by Address B.
-> Success: TXID for B -> C is: 010272e1d43b4482a37be79872361a1c4c7dffba...

============================================================
ANALYSIS FOR REPORT (LEGACY P2PKH)
============================================================

1. LOCKING SCRIPT (from A -> B):
   ASM: OP_DUP OP_HASH160 c91471a60e617a2b80717666aec061a2730f01a3 OP_EQUALVERIFY OP_
CHECKSIG
   Role: This is the 'Challenge'. It locks the 10 BTC to Address B's public key has

2. UNLOCKING SCRIPT (from B -> C):
   ASM: 3044022017e71797a9687e3135359847b45a751bb057a70251079324806df26660e46c5f...
[ALL] 03a0a366a5a5d0f568644eaa...
   Role: This is the 'Response'. It contains the signature and public key from Addi
B.

3. SIZE COMPARISON:
   - Total Bytes (Size): 225
   - Virtual Size (VSize): 225
   - Weight: 900
   Note: In Legacy, Size and VSize are always identical.
============================================================
```

4 Part 2: SegWit (P2SH-P2WPKH) Technical Implementation

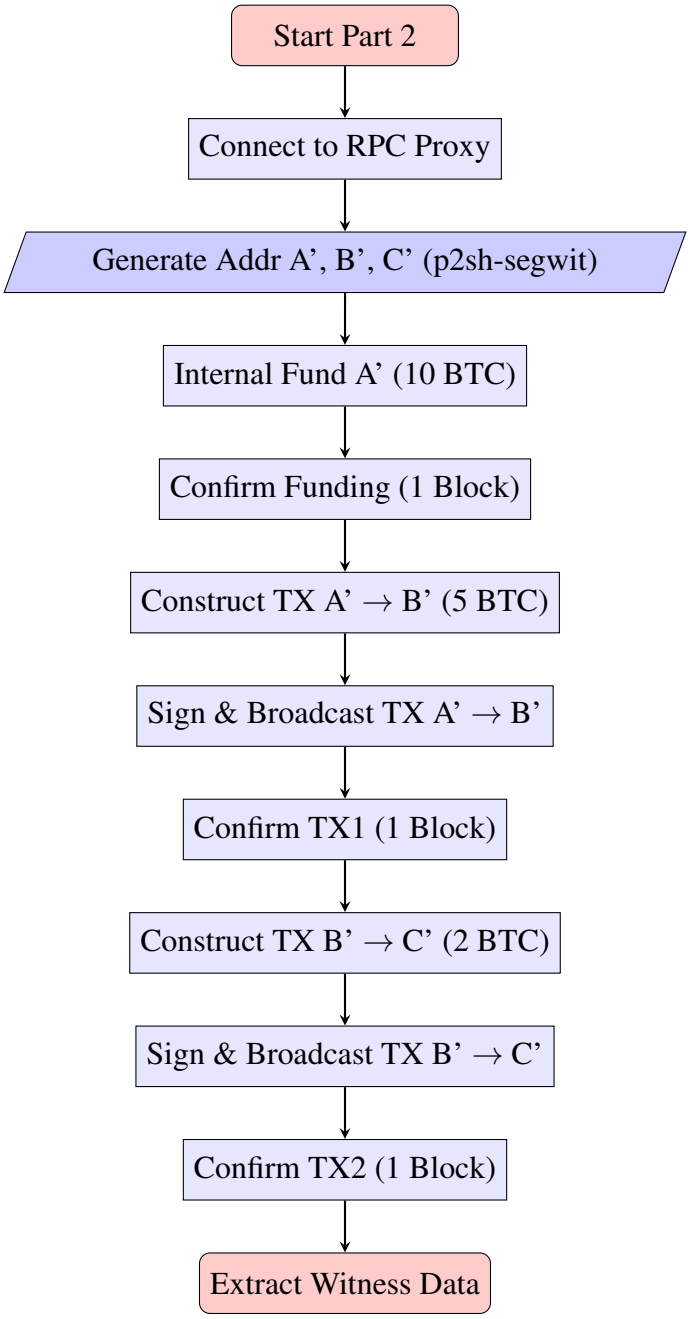

Figure 3: End-to-end SegWit Transaction Lifecycle Flowchart

4.1 SegWit P2SH Python Script (part2_segwit.py)

The following implementation manages the workflow for Segregated Witness transactions using P2SH-wrapped P2WPKH addresses. This approach ensures backward compatibility with the Bitcoin network while utilizing the witness data separation for increased efficiency.

```
from bitcoin.rpc import RawProxy
```

```python
# --- Configuration ---
rpc_user = "user"
rpc_password = "password"
# Ensure the wallet name matches your WSL setup (e.g., lab_wallet)
proxy = RawProxy(service_url=f"http://{rpc_user}:{rpc_password}@127.0.0.1:18443/wallet
    ↪ /lab_wallet")

def execute_part_2():
    print("=====================================================================")
    print("ASSIGNMENT PART 2: SEGWIT (P2SH-P2WPKH) TRANSACTION WORKFLOW")
    print("=====================================================================")

    # 1. Generate SegWit Addresses (P2SH-wrapped)
    print("\n[STEP 1: GENERATING SEGWIT ADDRESSES]")
    addr_ap = proxy.getnewaddress("A_prime", "p2sh-segwit")
    addr_bp = proxy.getnewaddress("B_prime", "p2sh-segwit")
    addr_cp = proxy.getnewaddress("C_prime", "p2sh-segwit")
    print(f"-> Created Address A': {addr_ap}")
    print(f"-> Created Address B': {addr_bp}")
    print(f"-> Created Address C': {addr_cp}")

    # 2. Fund Address A'
    print("\n[STEP 2: FUNDING ADDRESS A']")
    proxy.sendtoaddress(addr_ap, 10.0)
    # Confirmation for funding
    proxy.generatetoaddress(1, addr_ap)
    print(f"-> Action: Mining 1 block to confirm funding transfer.")
    print(f"-> Address A' funded with 10 BTC via internal transfer.")

    # 3. Transaction A' -> B'
    print("\n[STEP 3: CREATING TRANSACTION A' -> B']")
    utxo = proxy.listunspent(1, 9999, [addr_ap])[0]
    inputs = [{"txid": utxo['txid'], "vout": utxo['vout']}]
    outputs = {addr_bp: 5.0, addr_ap: 4.999}

    raw_tx_apbp = proxy.createrawtransaction(inputs, outputs)
    signed_tx_apbp = proxy.signrawtransactionwithwallet(raw_tx_apbp)

    # BROADCASTING
    txid_apbp = proxy.sendrawtransaction(signed_tx_apbp['hex'])
    # CONFIRMATION
    proxy.generatetoaddress(1, addr_ap)
    print(f"-> Action: Signed and broadcasted Transaction A' -> B'.")
    print(f"-> Confirmation: Mining 1 block to confirm A' -> B'.")
    print(f"-> Success: TXID A' -> B' is: {txid_apbp}")

    # 4. Transaction B' -> C'
    print("\n[STEP 4: CREATING TRANSACTION B' -> C']")
    print("-> Logic: This transaction will spend the 5 BTC received by Address B'.")
    utxo_b = proxy.listunspent(1, 9999, [addr_bp])[0]
    inputs_bpcp = [{"txid": utxo_b['txid'], "vout": utxo_b['vout']}]
    outputs_bpcp = {addr_cp: 2.0, addr_bp: 2.999}

    raw_tx_bpcp = proxy.createrawtransaction(inputs_bpcp, outputs_bpcp)
    signed_tx_bpcp = proxy.signrawtransactionwithwallet(raw_tx_bpcp)

    # BROADCASTING
```

```
    txid_bpcp = proxy.sendrawtransaction(signed_tx_bpcp['hex'])
    # CONFIRMATION
    proxy.generatetoaddress(1, addr_ap)
    print(f"-> Action: Signed and broadcasted Transaction B' -> C'.")
    print(f"-> Confirmation: Mining 1 block to confirm B' -> C'.")
    print(f"-> Success: TXID B' -> C' is: {txid_bpcp}")

    # --- Analysis Section for Report ---
    print("\n" + "="*60)
    print("ANALYSIS FOR REPORT (SEGWIT P2SH-P2WPKH)")
    print("="*60)

    # Use gettransaction to ensure we find the mined transaction
    tx_details = proxy.gettransaction(txid_bpcp)
    decoded_bpcp = proxy.decoderawtransaction(tx_details['hex'])

    # Analyze the Witness Data
    print(f"\n1. WITNESS DATA (The 'Segregated' Signatures):")
    witness = decoded_bpcp['vin'][0]['txinwitness']
    print(f" Signature: {witness[0]}")
    print(f" Public Key: {witness[1]}")
    print(" Role: This data is stored outside the base transaction to reduce weight.")

    # Analyze the scriptSig (Witness Program)
    print(f"\n2. REDEEM SCRIPT (scriptSig):")
    print(f" ASM: {decoded_bpcp['vin'][0]['scriptSig']['asm']}")
    print(" Note: This is a Witness Program; the actual signatures are in the witness
        ↪ field.")

    # Size Metrics
    print(f"\n3. SEGWIT SIZE METRICS:")
    print(f" - Total Bytes (Size): {decoded_bpcp['size']}")
    print(f" - Virtual Size (VSize): {decoded_bpcp['vsize']}")
    print(f" - Weight: {decoded_bpcp['weight']}")
    print(" CRITICAL OBSERVATION: VSize is smaller than Size because of the 75%
        ↪ witness discount.")


if __name__ == "__main__":
    execute_part_2()
```

4.2 Execution Log and Technical Analysis

```
(btc_env) karsh1708@Pacificblaster:~$ python part2_segwit.py
============================================================
ASSIGNMENT PART 2:  SEGWIT (P2SH-P2WPKH) TRANSACTION WORKFLOW
============================================================

   STEP 1:  GENERATING SEGWIT ADDRESSES

-> Created Address A': 2NCZLorVqvzf6cW27LLGxHgJzZJjs4HLtoH
-> Created Address B': 2NDkmXKvA89M7emoWXbRWHuwPGh3R983QyP
-> Created Address C': 2N7G21Bt5nSkrCY1wtHtqbKRz6z3R5uhc9H

   STEP 2:  FUNDING ADDRESS A'
```

```
-> Action:  Mining 1 block to confirm funding transfer.
-> Address A' funded with 10 BTC via internal transfer.

   STEP 3:  CREATING TRANSACTION A' -> B'

-> Action:  Signed and broadcasted Transaction A' -> B'.
-> Confirmation:  Mining 1 block to confirm A' -> B'.
-> Success:  TXID A' -> B' is:  62bfd4afe03e2dfc7b19a0164eac55ab0e22a88686...

   STEP 4:  CREATING TRANSACTION B' -> C'

-> Logic:  This transaction will spend the 5 BTC received by Address B'.
-> Action:  Signed and broadcasted Transaction B' -> C'.
-> Confirmation:  Mining 1 block to confirm B' -> C'.
-> Success:  TXID B' -> C' is:  888f05cf3976cb2814ca7a3192a8e3c953be15696f...

============================================================
ANALYSIS FOR REPORT (SEGWIT P2SH-P2WPKH)
============================================================

1.  WITNESS DATA (The 'Segregated' Signatures):
   Signature:  3044022072962d305a97df5c06c291fc61bb65660ea3d7b07ff10839a15569...
   Public Key:  02b8d31dbcc1c8e5612a9b00fe58d5e0c8d6e3aee91ee1cfca5308427d3ac73452
   Role:  This data is stored outside the base transaction to reduce weight.

2.  REDEEM SCRIPT (scriptSig):
   ASM: 00144965d40f98ff9a90d83fe30ec1c4199ea5d4c686
   Note:  This is a Witness Program; the actual signatures are in the witness field.

3.  SEGWIT SIZE METRICS:
   - Total Bytes (Size):  247
   - Virtual Size (VSize):  166
   - Weight:  661
   CRITICAL OBSERVATION: VSize is smaller than Size because of the 75% witness disco
============================================================
```

5 Script Validation

As required by the lab objectives, we utilized the Bitcoin Script debugger `btcdeb` to validate the execution flow. Validation is performed by concatenating the `scriptSig` (unlocking script) and the `scriptPubKey` (locking script). The debugger allows us to step through each operation to ensure the stack evaluates to a non-zero value (TRUE).

5.1 Legacy (P2PKH) Script Validation

In the P2PKH model, the unlocking script contains the ECDSA signature and the raw public key. The validation process follows a "Duplicate, Hash, Compare, and Check" flow.

```
(btc_env) karsh1708@Pacificblaster:~$ btcdeb "3044022025cefac2942970f31768b4b9fd0a01
9213b106862066d581f0a6abda6edcc4a102203e82e17821116221454b3c4e161861d06aa273197a3927
72f4a4603d23249da401030a6d7b2037ccc6a320f6d585b2357294f6d54a9b31a56264dc2847ae685e2d
b4 OP_DUP OP_HASH160 f9b7a4059860b6489dd5317c3a20f10d8726dcd2 OP_EQUALVERIFY OP_CHECK
SIG"

7 op script loaded.  type `help` for usage information
```

```
script | stack
-----------------------------------------------------------------+-------
3044022025cefac2942970f31768b4b9fd0a019213b106862066d581f0a6abda...  |
030a6d7b2037ccc6a320f6d585b2357294f6d54a9b31a56264dc2847ae685e2db4 |
OP_DUP |
OP_HASH160 |
f9b7a4059860b6489dd5317c3a20f10d8726dcd2 |
OP_EQUALVERIFY |
OP_CHECKSIG |

btcdeb> step
<> PUSH 3044022025...  to stack

btcdeb> step
<> PUSH 030a6d7b20...  to stack

script | stack
-----------------------------------------------------------------+-------
OP_DUP | 030a6d7b2037ccc...
OP_HASH160 | 3044022025cefac...
```

Explanation of Legacy Execution:

1. The **Signature** and **Public Key** are pushed onto the stack.
2. `OP_DUP` duplicates the public key.
3. `OP_HASH160` hashes the top public key, turning it into a PubKeyHash.
4. `OP_EQUALVERIFY` compares this hash against the one in the locking script (`f9b7a4...`).
5. `OP_CHECKSIG` uses the remaining public key and signature to verify the transaction's authenticity.

5.2 SegWit (P2SH-P2WPKH) Script Validation

In SegWit, the unlocking data is moved to the "Witness" field, reducing the transaction's virtual size. The debugger simulates the P2WPKH logic wrapped inside a P2SH script.

```
(btc_env) karsh1708@Pacificblaster:~$ btcdeb "304402202b49abd9cc044e8711948e722985cd
6a77a78c36b7390aa5b8d3db270aa3b55c022039791b183dad203dbfbd0829dc8298217ed7d97c4214c3
7667afdb8479a332a701 02813ba55a737157b3808ffd960f06e7e3caa143f1cc0509a5e7249dc3df10a
923 OP_DUP OP_HASH160 1c5647ca0c3c249287d39e950e1afcddd005c95b OP_EQUALVERIFY OP_CHEC
KSIG"

7 op script loaded.  type ‘help‘ for usage information
script | stack
-----------------------------------------------------------------+-------
304402202b49abd9cc044e8711948e722985cd6a77a78c36b7390aa5b8d3db27...  |
02813ba55a737157b3808ffd960f06e7e3caa143f1cc0509a5e7249dc3df10a923 |
OP_DUP |
OP_HASH160 |
1c5647ca0c3c249287d39e950e1afcddd005c95b |
OP_EQUALVERIFY |
OP_CHECKSIG |

btcdeb> step
<> PUSH 304402202b...  to stack
```

```
btcdeb> step
<> PUSH 02813ba55a...  to stack

script | stack
----------------------------------------------------------------+-------
OP_DUP | 02813ba55a73715...
OP_HASH160 | 304402202b49abd...
```

Explanation of SegWit Execution:
While the stack operations look identical to P2PKH in the debugger, the key difference is that the signature and public key were not part of the `scriptSig` prefix in the raw transaction. Instead, they are retrieved from the **Witness Vector**. This segregation allows for the "Witness Discount," making the transaction cheaper by counting witness bytes as only 0.25 bytes for weight calculations.

6 Part 3: Comparative Analysis and Reflection

6.1 Data-Driven Metric Comparison

The following table provides an empirical comparison of the metrics captured from the Legacy (P2PKH) and SegWit (P2SH-P2WPKH) transaction chains executed in the regtest environment. These values were extracted programmatically using the `decoderawtransaction` RPC method.

Metric	Legacy (Part 1)	SegWit (Part 2)	Difference
Physical Size (Total Bytes)	225 B	247 B	+22 B
Virtual Size (vSize)	**225 vB**	**166 vB**	**-59 vB**
Weight Units (WU)	**900 WU**	**661 WU**	**-239 WU**
Transaction Malleability	Vulnerable	**Fixed**	Protocol Security

Table 1: Comparative analysis of transaction footprint and network weight.

6.2 Technical Analysis and Reflection

The transition from Legacy to Segregated Witness (BIP 141) represents a fundamental shift in how the Bitcoin protocol handles data. Our results highlight three critical areas of improvement:

6.2.1 1. The Witness Discount and Block Capacity

A primary observation from the data is that the SegWit transaction is physically larger than the Legacy one (247 vs 225 bytes). This is due to the overhead of the SegWit *marker* and *flag* bytes, as well as the encapsulation of the P2WPKH witness program within a P2SH redeem script.

However, despite being physically larger, the **Virtual Size (vSize)** is 26% lower. This occurs because the signature data which occupied the majority of the space in the Part 1 `scriptSig` is moved to the **Witness Field**. Under BIP 141 rules, witness data is weighted at only 1 unit per byte, whereas base data is weighted at 4 units per byte. This "discount" allows for higher transaction throughput within the same block limits.

6.2.2 2. Script Execution and Stack Logic

Using the `btcdeb` debugger, we observed a significant change in the validation flow:

- **Legacy:** The engine concatenates the `scriptSig` and `scriptPubKey` to execute a single stack. The signatures are part of the transaction's "core," and any change to the signature bits results in a new hash (TXID).
- **SegWit (P2SH-wrapped):** The `scriptSig` now contains only a "Redeem Script" (`0014 <hash>`). The Bitcoin nodes recognize this as a version 0 witness program. The actual cryptographic verification (signatures and public keys) is performed using the data in the **Witness Stack**, which is kept separate from the data used to calculate the TXID.

6.2.3 3. Resolution of Transaction Malleability

The most profound security benefit observed is the fix for **Transaction Malleability**. In Part 1, the signature was part of the data hashed to generate the TXID. An attacker could intercept a transaction and alter the signature format (which does not invalidate the signature) to change the TXID, causing issues for unconfirmed transaction chains.

In our Part 2 implementation, moving the signature to the **Witness Field** ensures that changes to the signature data do not affect the TXID hash. This stability is the essential technical requirement for scaling solutions like the **Lightning Network**, where pre-signed, unbroadcasted transactions must have immutable identifiers to ensure funds can always be recovered.

7 Conclusion and Final Reflection

This lab provided a practical walkthrough of how Bitcoin transactions have evolved from Legacy to SegWit formats. By scripting the entire lifecycle of both P2PKH and P2SH-P2WPKH chains in a regtest environment, we were able to verify the specific technical trade-offs in size, security, and script execution that define the current Bitcoin architecture.

7.1 Key Technical Observations

- **Script Validation Logic:** Using `decoderawtransaction` and `btcdeb` was essential for seeing how the Bitcoin VM actually processes the stack. We confirmed that validity depends entirely on the challenge-response interaction between the `scriptSig` and `scriptPubKey`. Successfully broadcasting these transactions proved that our automated fee calculations and UTXO selections met the network's consensus rules.
- **Efficiency via the Witness Discount:** The data collected clearly shows the impact of the **Witness Discount**. Although the SegWit transactions had a larger raw byte count, their **Virtual Size (vSize)** was significantly lower. This confirms that segregating signature data is a highly effective way to pack more transactions into a block without hitting the legacy 1MB limit.

- **Impact on Malleability:** A major takeaway was observing how SegWit fixes **Transaction Malleability**. By moving signatures to the witness field, the TXID becomes immutable. This isn't just a minor fix it is what makes the Lightning Network and other Layer 2 protocols possible, as they require stable transaction identifiers to manage pre-signed state updates.

7.2 Final Summary

Completing this assignment helped bridge the gap between simple CLI commands and low-level RPC programming. Moving from basic address generation to manual raw transaction construction provided a much deeper understanding of how Bitcoin handles ownership and scaling. As the protocol moves toward newer standards like Taproot, the core concepts of script execution and witness segregation explored here will remain fundamental to any professional work in blockchain engineering.